

Architettura dei Calcolatori

Secondo Appello – 4 luglio 2022 – 1h45

1. [7 pt] Implementare in assembly RISC-V la funzione C **strncpy**

```
char *strncpy(char *dest, const char *src, size_t n);
```

che prende in ingresso una stringa destinazione **dest**, una stringa sorgente **src** e un numero di bytes **n**. La funzione copia al più **n** caratteri di **src** in **dest**, terminandola col terminatore **'\0'**. Se **n > strlen(src)** allora **src** verrà interamente copiata in **dst**.

Gestire correttamente lo stack come da calling convention, così come il passaggio dei parametri di ingresso e di uscita sui registri appropriati. Scrivere la funzione main che invoca **strncpy**, mostrando anche l'allocazione degli array sul segmento dati statico del file ELF.

- Main e dati 1pt
- 1pt per la gestione dello stack
- 2pt per la corretta gestione di `strlen < n`
- 3pt per la corretta implementazione della copia principale

SOLUZIONE `// a0 = dest; a1 = src; a2 = n;`

```
strncpy:
    addi sp,sp,-24          // adjust stack for 3 dw
    sd    s0,0(sp)          // push s0
    sd    s1,8(sp)          // push s1
    sd    s2,16(sp)         // push s2

    li    s2,0              // i = 0

L1:   add    s0,s2,a1        // s0 = addr of src[i]
    lbu    s0,0(s0)         // s0 = src[i]
    add    s1,s2,a0         // s1 = addr of dest[i]
    sb     s0,0(s1)         // dest[i] = src[i]
    beq    s0,zero,L3       // if src[i] == 0 then exit
    addi   s2,s2,1          // i = i + 1
    beq    s2,a2,L2         // if (i == n) then exit
    j      L1              // next iteration of loop

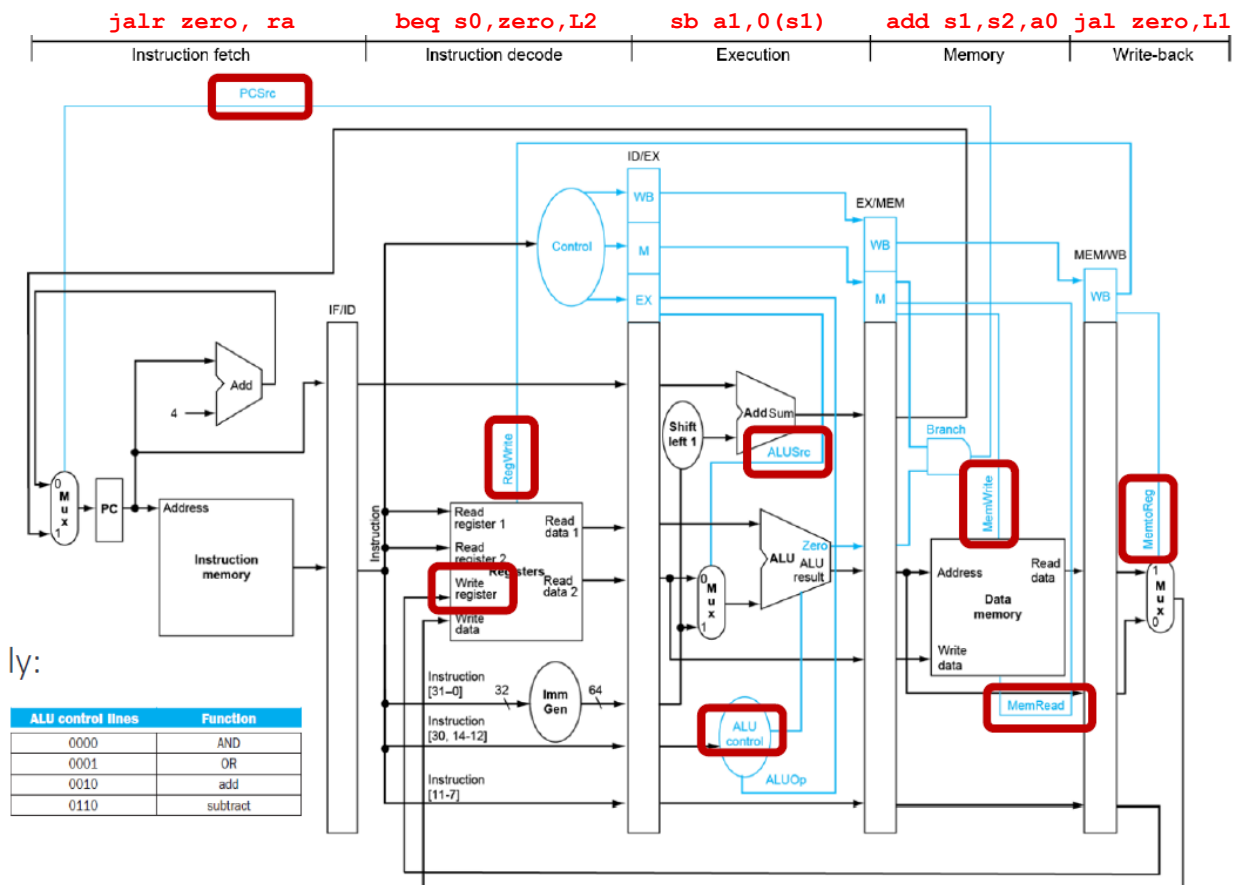
L2:   sb     zero,1(s0)      // dest[strlen+1] = '\0'

L4:   ld     s0,0(sp)        // restore saved s0
    ld     s1,8(sp)         // restore saved s1
    ld     s2,16(sp)        // restore saved s2
    addi   sp,sp,24         // pop 3 dw from stack
    ret                                // and return
```

2. Si consideri una cache che usa 4 bit per il campo offset e 4 bit per il campo index. Ipotizzando di aver già letto l'indirizzo 0x0000, quali affermazioni sono vere (spiegare il ragionamento per arrivare alla risposta)?
- a) Se eseguire letture agli indirizzi 0x0114, 0x0308, 0x1005 produce tutte conflict miss allora la cache è direct mapped
 - b) Se la stessa sequenza di letture del punto a) produce tutte cold cache miss allora la cache è set-associative, con almeno 4 vie
 - c) Se la stessa sequenza di letture del punto a) produce due cold cache miss e una conflict miss allora la cache è 2-way set-associative
 - d) Se la cache è 2-way set-associative la sua dimensione è 1KB
-
- a) **FALSA.** L'indirizzo 0x0114 ha campo index diverso dagli altri, coi dati forniti non è DM.
 - b) **VERA.** Dal momento che avevamo già letto 0x0000, una via del set con indice 0 è già occupata. Se gli indirizzi 0x0308 e 0x1005, che condividono lo stesso set, producono cold cache miss ci devono essere almeno altre due vie, quindi minimo 4-way (3-way non esiste).
 - c) **VERA.** La cold cache miss è quella dell'indirizzo 0x0114, che sfrutta un set diverso. Per i motivi detti al punto b, se gli indirizzi 0x0308 e 0x1005, che condividono lo stesso set, producono una cold cache miss (uso l'altra via) e poi una conflict miss è una 2-way.
 - d) **FALSA.** Con 4 bit di offset ogni linea usa 16 bytes, e con 4 bit di index ho 16 set. Ogni set ha 2 vie (quindi 2 linee per set) da cui ricavo la dimensione della cache: $16 \text{ set} * 32 \text{B/set} = 512 \text{B}$.

3. [5 pt] Si consideri il programma assembly RISC-V dell'esercizio 4. Considerando il diagramma della pipeline RISC-V single-cycle indicare quanto valgono i segnali di controllo cerchiati in figura quando l'ultima istruzione, la **jalr zero, ra**, si trova in fase di fetch.
- ATTENZIONE:** Si deve considerare il flusso di esecuzione dinamico del programma, non la sequenza statica di istruzioni. Si ragioni su quali istruzioni vengono eseguite prima della **jalr** per riempire la pipeline.

Punto bonus per la corretta impostazione del problema (pipeline riempita correttamente).
Fino a 4 punti per i segnali corretti.



PCSrc = 0 (nella fase di MEM c'è una ADD)
 RegWrite = 0 (nella fase di WB c'è una JAL)
 Write register = XX
 ALUSrc = 1 (nella fase di EXE c'è una SB)
 ALUctrl = 0010
 MemWrite = 0 (nella fase di MEM c'è una ADD)
 MemRead = 0
 MemtoReg = X (nella fase di WB c'è una JAL)

4. [8 pt] Dato il seguente stralcio di codice assembly RISC-V

```
L1:    add    s1,s2,a0
        sb    a1,0(s1)
        beq   s0,zero,L2
        addi  s2,s2,1
        jal   zero,L1
L2:    jalr   zero,ra
```

Si produca l'equivalente codice binario. Dove sono implicate istruzioni di alterazione del control flow si trasformino le label nei corrispondenti offset di salto.

Prima di tutto convertiamo gli alias dei registri nei loro indici reali, e poi convertiamo le label nei relativi offset.

```
add     x9,x18,x10
sb      x11,0(x9)
beq     x8,x0,0x006    // 3 istruzioni in avanti, con >> 1 = +6 → 0x006
addi    x18,x18,1
jal     x0,L1          // 4 istruzioni indietro, con >> 1 = -8 → ..1 0111+1=..1 1000 = 0xFFFF8
jalr    x0,0(x1)
```

0000 0000 1010 1001 0000 0100 1011 0011	0x00A904B3	1pt
0000 0000 1011 0100 1000 0000 0010 0011	0x00B48023	1pt
0000 0000 0000 0100 0000 0110 0110 0111	0x00040667	2pt
0000 0000 0001 1001 0000 1001 0001 0011	0x00190913	1pt
1111 1111 0001 1111 1111 0000 0110 1111	0xFF1FF06F	2pt
0000 0000 0000 0000 1000 0000 0110 0111	0x00008067	1pt

0000 0000 1001 0101 1000 0000 0010 0011 0x00958023 sb con registri invertiti

5. [5 pt] Si consideri di eseguire il seguente stralcio di codice su una pipeline RISC-V con logica di forwarding e con dynamic branch predictor a due bit inizialmente settato a BRANCH TAKEN.

```
li      t2,3 // UB
li      t0,0 // i = 0
loop_i
li      t1,0 // j = 0
loop_j
addi    t1,t1,1
blt     t1,t2,loop_j
addi    t0,t0,1
blt     t0,t2,loop_i

jr      ra
```

a) Si calcoli la misprediction rate [2,5 pt].

b) Sapendo che:

- per ogni predizione corretta un'istruzione di tipo branch impiega 2 cicli ad eseguire, mentre in caso di misprediction ne impiega 8
- le istruzioni aritmetico-logiche impiegano 2 cicli a eseguire
- il salto incondizionato impiega 8 cicli ad eseguire

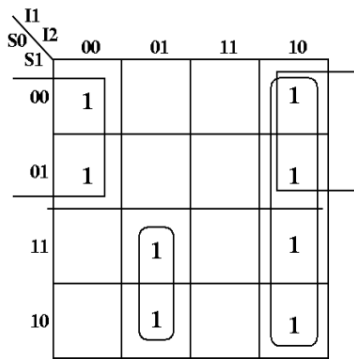
Si calcoli il CPI medio per l'esecuzione del programma [2,5 pt]

	istanza di istruzione	registri e condizioni	esito salto	2bit pred		1bit pred		miss#
1)	li t0, 0	t0 = 0						
2)	li t2, 3	t2 = 3						
3)	li t1, 0	t1 = 0						
4)	addi t1, t1, 1	t1 = 1						
5)	blt t1, t2, loop_j	t1 < t2 → 1 < 3	T	Ts	H	T	H	[1]
6)	addi t1, t1, 1	t1 = 2						
7)	blt t1, t2, loop_j	t1 < t2 → 2 < 3	T	Ts	H	T	H	[2]
8)	addi t1, t1, 1	t1 = 3						
9)	blt t1, t2, loop_j	t1 < t2 → 3 < 3	N	Ts	M	T	M	[3]
10)	addi t0, t0, 1	t0 = 1						
11)	blt t0, t2, loop_i	t0 < t2 → 1 < 3	T	Tm	H	N	M	[4]
12)	li t1, 0	t1 = 0						
13)	addi t1, t1, 1	t1 = 1						
14)	blt t1, t2, loop_j	t1 < t2 → 1 < 3	T	Ts	H	T	H	[5]
15)	addi t1, t1, 1	t1 = 2						
16)	blt t1, t2, loop_j	t1 < t2 → 2 < 3	T	Ts	H	T	H	[6]
17)	addi t1, t1, 1	t1 = 3						
18)	blt t1, t2, loop_j	t1 < t2 → 3 < 3	N	Ts	M	T	M	[7]
19)	addi t0, t0, 1	t0 = 2						
20)	blt t0, t2, loop_i	t0 < t2 → 2 < 3	T	Tm	H	N	M	[8]
21)	li t1, 0	t1 = 0						
22)	addi t1, t1, 1	t1 = 1						
23)	blt t1, t2, loop_j	t1 < t2 → 1 < 3	T	Ts	H	T	H	[9]
24)	addi t1, t1, 1	t1 = 2						
25)	blt t1, t2, loop_j	t1 < t2 → 2 < 3	T	Ts	H	T	H	[10]
26)	addi t1, t1, 1	t1 = 3						
27)	blt t1, t2, loop_j	t1 < t2 → 3 < 3	N	Ts	M	T	M	[11]
28)	addi t0, t0, 1	t0 = 3						
29)	blt t0, t2, loop_i	t0 < t2 → 3 < 3	N	Tm	M	N	H	[12]
30)	j r ra							

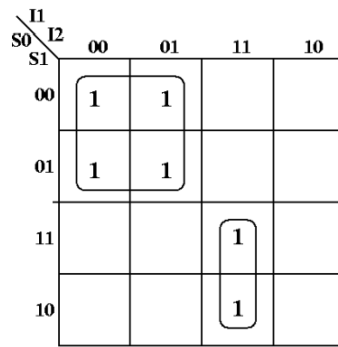
Su 30 istruzioni totali 12 sono branch, 17 aritmetiche (CPI=2) e 1 salto incondizionato (CPI=8).
Per il singolo predittore la misprediction rate è 4/12. Quindi il CPI medio per il programma è

$$\begin{aligned}
 CPI &= \frac{17 I_{ALU} * 2 \frac{\text{cicli}}{I_{ALU}} + 1 I_{JMP} * 8 \frac{\text{cicli}}{I_{JMP}} + 12 I_{BR} \left(\frac{4 \text{ mis} * 8 \frac{\text{cicli}}{\text{mis}} + 8 \text{ pr}_{ok} * 2 \frac{\text{cicli}}{\text{pr}_{ok}}}{12} \right)}{30 I} \\
 &= \frac{34 + 8 + 12 \left(\frac{32 + 16}{12} \right)}{30} = \frac{90}{30} = 3
 \end{aligned}$$

[7 pt] Date le mappe di Karnaugh riportate in figura



$$S1^* = \sim S0 \cdot \sim I2 + I1 \cdot I2 + S0 \cdot \sim I1 \cdot I2$$

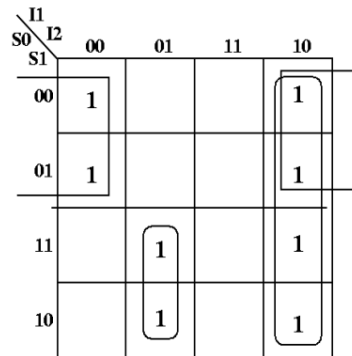


$$S0^* = \sim S0 \cdot \sim I1 + I1 \cdot I2 \cdot S0$$

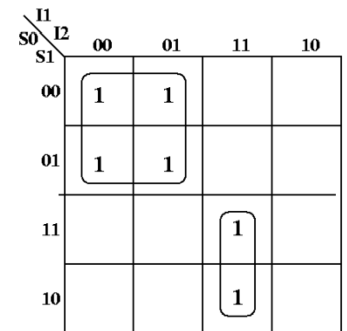
- c) [3,5 pt] derivare a ritroso la tabella di verità, e da lì il diagramma di transizione degli stati.
Dire se si tratta di un automa di Mealy o di Moore;
- d) [2 pt] trovare le forme minime della rete logica sequenziale descritta dalle mappe;
- e) [1,5 pt] disegnare il circuito finale.

Tabelle di verità per NextState

S0	S1	I1	I2	S0*	S1*
0	0	0	0	1	1
0	0	0	1	1	0
0	0	1	0	0	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	1
1	0	1	0	0	1
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	1	0



$$S1^* = \sim S0 \cdot \sim I2 + I1 \cdot I2 + S0 \cdot \sim I1 \cdot I2$$



$$S0^* = \sim S0 \cdot \sim I1 + I1 \cdot I2 \cdot S0$$

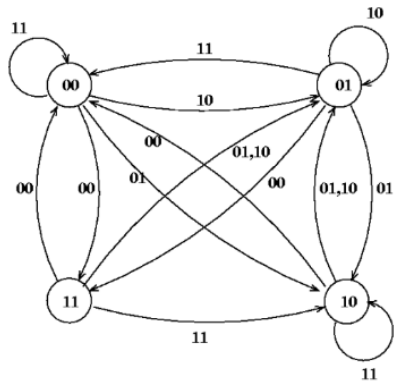
Circuito finale

$$S1^* = \sim S0 \cdot \sim I2 + I1 \cdot I2 + S0 \cdot \sim I1 \cdot I2$$

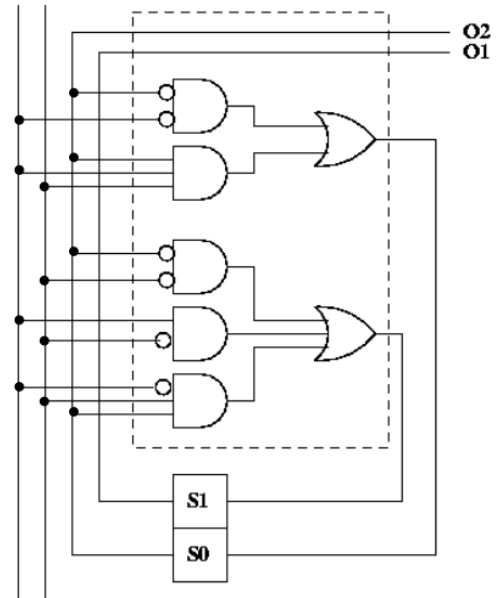
$$S0^* = \sim S0 \cdot \sim I1 + I1 \cdot I2 \cdot S0$$

$$O1 = S0$$

$$O2 = S1$$



I1 I2



Appendice: registri RISC-V

Register(s)	Alt.	Description
x0	zero	The zero register, always zero
x1	ra	The return address register, stores where functions should return
x2	sp	The stack pointer, where the stack ends
x5-x7, x28-x31	t0-t6	The temporary registers
x8-x9, x18-x27	s0-s11	The saved registers
x10-x17	a0-a7	The argument registers, a0-a1 are also return value